

Trioron 1.1: a multi-axis substrate for resource-bounded continual learning

Marcelinus R. Hatorangan*

May 24, 2026

Abstract

We describe *Trioron 1.1*, a substrate-level revision to the trioron architecture introduced in the companion paper, exposing six per-cell axes of structural plasticity: long-range input reach, plastic fanout sparsity, depth growth, slow axonal gain, dendritic compartmentalization, and field-conditional layer emergence. The six axes share one design principle — each is a separate write channel into the substrate, with its own provenance and its own plasticity gate, rather than a feature of the loss or the optimizer. A small spatial primitive (positions on a unit square, partitioned into pools) then enables a cell-granularity composition operator: two organisms trained independently on disjoint task slices can be absorbed into one substrate without retraining when they share the L0 perception layer’s deterministic factor (the R·S handshake). The cell-granularity composition is compatible with positional locality constraints (locally-connected masks) because absorption respects pools, not exact positions. We report the mechanism’s first verification on a chained-15 disjoint-class slice: task-aware accuracy is preserved on both donors within $\Delta \leq 0.026$, full-softmax accuracy lifts by +0.20 to +0.31 via manifold merge plus a 200-step head-only calibration pass, and the post-absorption substrate is *more* locality-clean than either donor alone. The 1.1 substrate is backwards-compatible with 1.0 donors: legacy checkpoints load unchanged and run under the extended forward path with the new axes at their default-off settings.

1 Introduction

2 Related Work

3 Methods

3.1 The Trioron Node, recapitulated

3.2 Axis 1: Long-range input reach

3.3 Axis 2: Plastic fanout sparsity

3.4 Axis 3: Depth growth

The function `insert_layer(net, after_idx)` splices a new `TrioronLayer` between layers `after_idx` and `after_idx+1`. The inserted layer is identity-initialized in the style of `Net2Net`, with optional

*Correspondence: marcelinusrocky@aol.com. Repository: <https://github.com/Marcrockhat/trioron-project>. Live demo: huggingface.co/spaces/Marcrockhat/trioron-demo. Guided tour: marcrockhat.github.io/trioron-project/tour.

Gaussian noise, so it begins as a pass-through and the network’s input–output map is unchanged at the moment of insertion. A per-slot cap `K_insert` prevents unbounded depth growth at any single position.

Depth insertion is gated by the `allow_insert_layer` flag on the active `TrioronProfile`. It defaults to OFF in the OPEN, CLASSIFICATION, and EDGE profiles and is enabled only in REASONING, because pairwise ablation on the chained-15 benchmark showed that depth growth regresses accuracy when combined with Axes 4 and 5. The intended usage pattern is a one-shot depth injection at a curriculum boundary, not continuous insertion during training.

3.5 Axis 4: Slow axonal gain

Each node carries a per-node multiplicative scalar `axonal_gain` that scales its outgoing edge contributions, giving an effective weight $W_{\text{eff}}[i, :] = \text{axonal_gain}[i] \cdot W[i, :]$. The gain operates on a slower timescale than ordinary weight learning: it is not updated by backpropagation but is instead written by external signals—reward, attention, manual priors—via `set_axonal_gain(net, layer_idx, gains)`. EWC anchors the gain alongside W , so consolidated gain values resist drift on subsequent tasks.

The purpose is task-dependent up- or down-regulation of a cell’s influence without modifying the learned weights themselves, analogous to neuromodulatory gain control in cortical circuits.

3.6 Axis 5: Dendritic compartmentalization

Each cell can host $K \geq 1$ dendritic branches, controlled by the profile parameter `B_max`. A flat `branch_id` vector partitions the fan-in columns into K groups; each branch computes a local dot-product over its partition and passes the result through a per-branch nonlinearity σ_{branch} (default `quad`: $\sigma(x) = x + x^2$, an NMDA-style supralinear activation) before soma-level pooling via learned `branch_weight`. When $K = 1$ the forward pass is byte-identical to v1.0 (a plain `F.linear` call). New branches are added with `grow_branch`; `inherit_dendrite` seeds a child branch from a parent’s partition for sister-specialist differentiation.

Phase 6 falsification confirmed that dendritic compartmentalization is load-bearing: $K=2$ lifts $K=1$ by +0.49 absolute accuracy on a concentric-ring discrimination task and by +0.76 absolute on a four-class variant, with the trigger arm matching forced-branch performance within noise.

3.7 Axis 6: Field-conditional layer emergence

Two per-node field channels drive conditional cell emergence. `epi_A` tracks chronic neighborhood frustration: a Gaussian diffusion of each cell’s `internal_stress` over the spatial positions of nearby cells. `epi_B` tracks self-activity as an exponential moving average. The spawn rule `axis6_spawn` selects candidates by a relative criterion: cells whose neighbors are stressed (high `epi_A`) but that are themselves quiet (low `epi_B`) become spawn sites. New cells inherit position from their parent plus Gaussian jitter, so they appear near the high-stress region rather than at arbitrary locations.

After each task, cells whose engagement fraction exceeds a threshold earn “credit” and have their gradients zeroed on all future tasks—a credit-based row-freezing mechanism that is strictly stronger than EWC’s quadratic penalty, because frozen rows experience zero drift rather than penalized drift.

3.8 Spatial substrate: positions, pools, and locality

Every cell carries a 3D continuous coordinate `cell_position`, assigned during a developmental phase from the positions of upstream cells. A `pool_id` partitions the $[0, 1]^2$ plane into a grid (default

$4 \times 4 = 16$ pools) used for pool-matched absorption (§3.9).

Locally-Connected masks (`enable_lcn`) optionally tie each cell’s weight row to a spatial window of input cells. The mask can be a soft Gaussian ($\exp(-d^2/2\sigma^2)$) or a hard top- K nearest-neighbor selection; it is applied element-wise to W during the forward pass so that off-window weights stay zero. When cells are added via `grow_node`, `extend_lcn_mask` automatically extends the mask to cover the new topology. LCN defaults to off, preserving the substrate’s natively flat-dense character; it is enabled per-layer or per-profile when spatial locality is beneficial.

3.9 Pool-matched absorption

4 Results

4.1 Substrate self-tests: per-axis end-to-end

4.2 Pool-matched absorption: the LCN unlock

4.3 Backward compatibility

5 Discussion

6 Conclusion

A Detailed Methods

A.1 Locally-Connected mask construction

A.2 Pool partition

A.3 Pool-matched absorption procedure

A.4 Manifold archive

B Reproducibility